로그인 페이지 작업 정리

- **브랜치:** `feat/skyu`
- **커밋:** `c0ed5aa` 일부 계정 로그인 가능 및 세부 기능 구현
- **작업일:** 2026-10-16

1. 작업 요약

구분	내용	상태
UI	로그인 타이틀 · 쿠폰 혜택 안내 문구 추가	완료
UI	혜택 영역 하단 `(일부 상품 제외)` 문구 추가	완료
보안	Google reCAPTCHA v2 Invisible 적용	완료
기능	아이디/비밀번호 로그인 (Server Action + 세션 쿠키)	완료
기능	아이디 저장 (localStorage)	완료
기능	네이버/카카오 로그인 버튼 → `<a href>` 링크	임시 주소
기능	로그아웃 버튼, 헤더 로그인 상태 표시, 회원가입	미구현

2. 로그인 페이지 UI

파일: `app/(shop)/login/page.tsx`, `app/(shop)/login/LoginLinks.tsx`

스타일은 Tailwind 유틸리티 클래스로 작성했습니다.

2-1. 상단 타이틀 · 안내 문구

- `로그인` 타이틀 (`text-[34px] font-bold`, 가운데 정렬)
- `MYSHOP이 처음이시라면 최대 20% 쿠폰 혜택을 드립니다. 자세히`
- `20%` 는 빨간색 (`text-red-600`)
- `자세히`는 회색 밑줄 링크 → 하단 혜택 영역(`#benefits`)으로 스크롤

2-2. 페이지 배치 순서

1. 로그인 타이틀 · 안내 문구
2. 아이디 / 비밀번호 입력, 아이디 저장, 로그인 버튼
3. 네이버 / 카카오 로그인 버튼
4. 비회원 주문조회 · 아이디 찾기 · 비밀번호 찾기
5. **reCAPTCHA 배지** (가운데 정렬, 위아래 30px 간격)
6. 통합 회원가입 버튼
7. 회원 혜택 영역 + `(일부 상품 제외)` (우측 정렬, 위쪽 구분선)

2-3. 로그인 폼 동작

- 로그인 실패 시 비밀번호 입력란 아래에 빨간 에러 메시지 표시
- 로그인 요청 중에는 버튼이 `로그인 중...`으로 바뀌고 비활성화 (중복 클릭 방지)
- `아이디 저장` 체크 시 아이디를 브라우저(localStorage)에 저장 → 다음 방문 시 자동 입력

3. reCAPTCHA

3-1. 방식

reCAPTCHA v2 Invisible + 인라인 배지를 사용합니다.

- 체크박스("로봇이 아닙니다") 없이 로그인 버튼을 누르면 백그라운드에서 검사
- 배지는 화면 오른쪽 아래 고정이 아니라 `아이디 찾기` 와 `통합 회원가입` 사이에 표시

3-2. 동작 흐름

1. 페이지에서 `https://www.google.com/recaptcha/api.js?render=explicit` 스크립트 로드
2. 로그인 버튼 클릭 → `grecaptcha.execute()` 로 토큰 발급
3. 토큰을 아이디/비밀번호와 함께 서버로 전송
4. 서버에서 secret key로 Google에 토큰 검증 (`domains/auth/lib/recaptcha.ts`)
5. 토큰은 1회용이므로 요청 후 `grecaptcha.reset()`

3-3. 키 관리

- 키는 reCAPTCHA 관리 콘솔에서 **v2 → Invisible reCAPTCHA badge**로 발급
- 도메인에 `localhost` 등록 필요 (배포 시 실제 도메인 추가)
- **개발 환경에서 site key가 없으면 reCAPTCHA를 건너뜀** → 키 없이도 로그인 개발 가능

4. 로그인 기능 (인증)

DB가 없는 현재 단계에서 Next.js만으로 로그인이 동작하도록 구현했습니다.

4-1. 전체 흐름

1. 사용자가 아이디/비밀번호 입력 후 로그인 클릭
2. 브라우저가 reCAPTCHA 토큰 발급
3. Server Action `login` 호출 (서버에서 실행)
4. reCAPTCHA 토큰 검증
5. `data/users.json`에서 사용자를 찾고 비밀번호 해시 비교
6. 성공 → 세션 쿠키(JWT) 발급 후 메인(`/`)으로 이동
7. 실패 → `아이디 또는 비밀번호가 올바르지 않습니다.` 메시지 표시

4-2. 파일 구성

파일	역할
`domains/auth/actions.ts`	Server Action `login`, `logout`
`domains/auth/dal.ts`	`getCurrentUser()` - 현재 로그인 사용자 조회 (비로그인 시 `null`)
`domains/auth/lib/session.ts`	세션 생성 · 조회 · 삭제 (JWT, `jose`)
`domains/auth/lib/users.ts`	사용자 조회, 비밀번호 검증 (scrypt)
`domains/auth/lib/recaptcha.ts`	reCAPTCHA 토큰 서버 검증
`data/users.json`	임시 사용자 저장소

4-3. 세션 쿠키

옵션	값	이유
`httpOnly`	`true`	브라우저 JS에서 쿠키 접근 불가 (XSS 방어)
`secure`	운영 환경에서만 `true`	로컬(http)에서도 동작하도록
`sameSite`	`lax`	다른 사이트에서의 요청 위조(CSRF) 완화
유효기간	7일	

4-4. 비밀번호 저장 방식

비밀번호는 평문이 아닌 **scrypt 해시**로 저장합니다.

```
"passwordHash": "<salt 32자리>:<hash 128자리>"
```

- **salt:** 비밀번호 설정 시 만든 랜덤 값 (같은 비밀번호라도 해시가 달라짐)
- **hash:** `scrypt(비밀번호, salt)` 결과
- 해시는 단방향이라 **해시값으로 원래 비밀번호를 알아낼 수 없습니다.**
- 로그인 시 입력한 비밀번호를 같은 salt로 다시 해시해 저장된 값과 비교합니다.

새 비밀번호 해시 만들기 (프로젝트 폴더에서 실행, `newPassword123` 부분을 원하는 비밀번호로 변경):

```
node -e "const c=require('crypto');const s=c.randomBytes(16).toString('hex');console.log(s+':'+
c.scryptSync('newPassword123',s,64).toString('hex'))"
```

출력된 값을 `data/users.json`의 `passwordHash`에 따옴표(`"`)로 감싸서 넣으면 됩니다.

4-5. 다른 페이지에서 로그인 사용자 쓰기

```
import { getCurrentUser } from '@/domains/auth/dal';

const user = await getCurrentUser(); // { id, loginId, name } 또는 null
```

- 서버 컴포넌트 / Server Action에서만 사용 가능
- 이 프로젝트는 `cacheComponents`가 켜져 있으므로, 쿠키를 읽는 컴포넌트는 반드시 `<Suspense>` 안에 둬야 합니다.

5. 네이버 / 카카오 로그인

- 버튼을 `<button>`에서 `<a href>`로 변경 (스타일 동일)
- 현재 주소는 **네이버/카카오 자체 로그인 페이지(임시)**라서, 눌러도 우리 사이트에 로그인되지 않습니다.
- 실제 소셜 로그인은 OAuth 흐름이 필요합니다.
- 네이버/카카오 로그인 · 동의
- 우리 사이트의 redirect URI로 `code` 전달
- 서버에서 `code` + client secret으로 토큰 교환
- 사용자 정보 조회 후 로그인 처리
- 네이버 Developers / 카카오 Developers에서 앱 등록 후 `page.tsx` 상단 `NAVER_LOGIN_URL`, `KAKAO_LOGIN_URL` 값을 교체하면 됩니다.

6. 팀원 환경 설정

6-1. 패키지 설치

pull 후 새 패키지(`jose`, `server-only`) 설치가 필요합니다.

```
npm install
```

6-2. `.env.local` 생성

프로젝트 루트(`package.json`과 같은 위치)에 `.env.local` 파일을 만듭니다. **값은 개인 채널로 공유하고, 저장소에는 절대 올리지 않습니다.** (`.gitignore`에 `.env*` 포함)

```
NEXT_PUBLIC_RECAPTCHA_SITE_KEY=
RECAPTCHA_SECRET_KEY=
SESSION_SECRET=
```

변수	설명	없을 때
`NEXT_PUBLIC_RECAPTCHA_SITE_KEY`	reCAPTCHA site key (공개용)	개발 환경에서는 reCAPTCHA 생략
`RECAPTCHA_SECRET_KEY`	reCAPTCHA secret key (서버 전용)	site key가 있으면 로그인 실패
`SESSION_SECRET`	세션 서명용 랜덤 문자열	**로그인 시 에러**

- `NEXT_PUBLIC_`로 시작하는 변수는 브라우저 코드에 포함되므로 **secret 값에는 절대 붙이지 않습니다.**
- 환경변수를 바꾼 뒤에는 `npm run dev`를 **재시작**해야 적용됩니다.

6-3. 테스트 계정

아이디	비밀번호
`asdf1234`	`test1234!`

`data/users.json`은 저장소에 커밋되므로 **테스트 계정만** 넣고 실제 사용자 정보는 넣지 않습니다.

7. 트러블슈팅

증상	원인	해결
`Missing required parameters: sitekey`	site key 미설정	`.env.local`에 키 추가 후 dev 서버 재시작 (현재는 키가 없으면 reCAPTCHA를 건너뛰도록 수정됨)
Hydration 에러 (`href` 값이 서버/클라이언트 다름)	코드 수정 후 브라우저의 JS가 이전 버전	강력 새로고침(Ctrl+Shift+R) → 안 되면 dev 서버 재시작 → 그래도 안 되면 `.next` 삭제 후 재시작
로그인 시 서버 에러	`data/users.json` 문법 오류 (예: 문자열 값의 따옴표 누락)	JSON 문자열 값은 반드시 `"..."`로 감싸기
`SESSION_SECRET 환경변수가 설정되지 않았습니다.`	`.env.local`에 `SESSION_SECRET` 없음	값 추가 후 dev 서버 재시작

8. 남은 작업

☐ 로그아웃 버튼 (`logout` Server Action은 준비됨)
☐ 헤더에 로그인 사용자 표시 (`getCurrentUser()` 사용, `<Suspense>` 필요)
☐ 회원가입 페이지 (`/member/join`)
☐ 실제 DB 연결 (`domains/auth/lib/users.ts`만 교체)
☐ 네이버 / 카카오 OAuth 연동
☐ 로그인이 필요한 페이지 접근 제한
☐ `develop` 최신 반영 후 PR 생성 (base: `develop`)